



Computer Organization and Architecture

CHAPTER 3

CPU

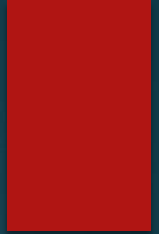
Learning Outcomes

- ▶ By the end of this chapter/ module student should be able to:
 - Explain the purpose of the Central Processing Unit (CPU)
 - Describe the components of the CPU and their functions
 - Discuss the fetch-execute cycle
 - Explain the Von Neumann architecture
 - Explain hardware and software interrupt
 - Give details on how the cache size, clock speed and number of cores affect CPU performance

Program Concept

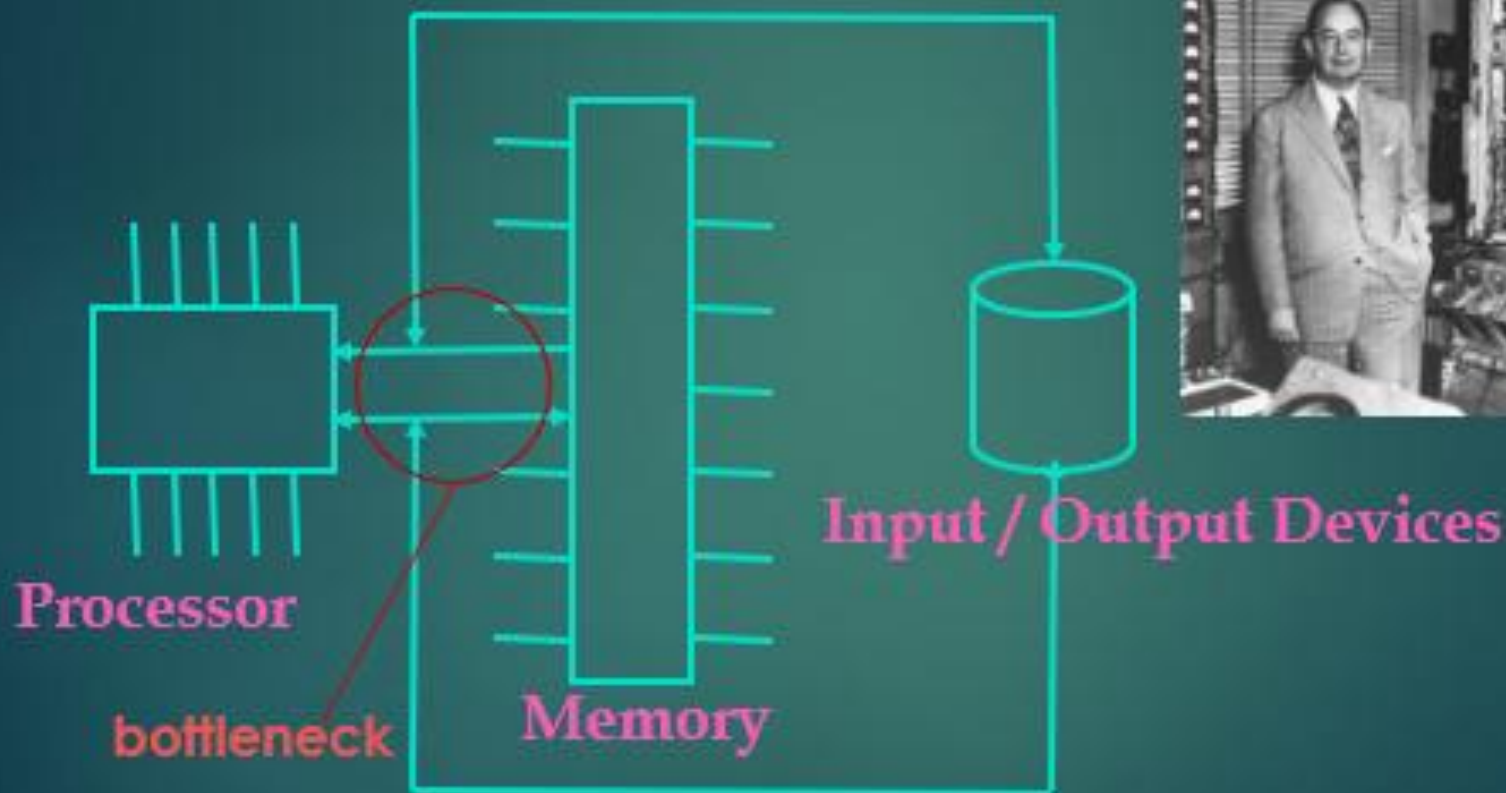
- ▶ Hardwired systems are inflexible
- ▶ General purpose hardware can do different tasks, given correct control signals
- ▶ Instead of re-wiring, supply a new set of control signals

What is a program?



- ▶ A sequence of steps
- ▶ For each step, an arithmetic or logical operation is done
- ▶ For each operation, a different set of control signals is needed

Von Neumann Model or “Princeton Architecture”.

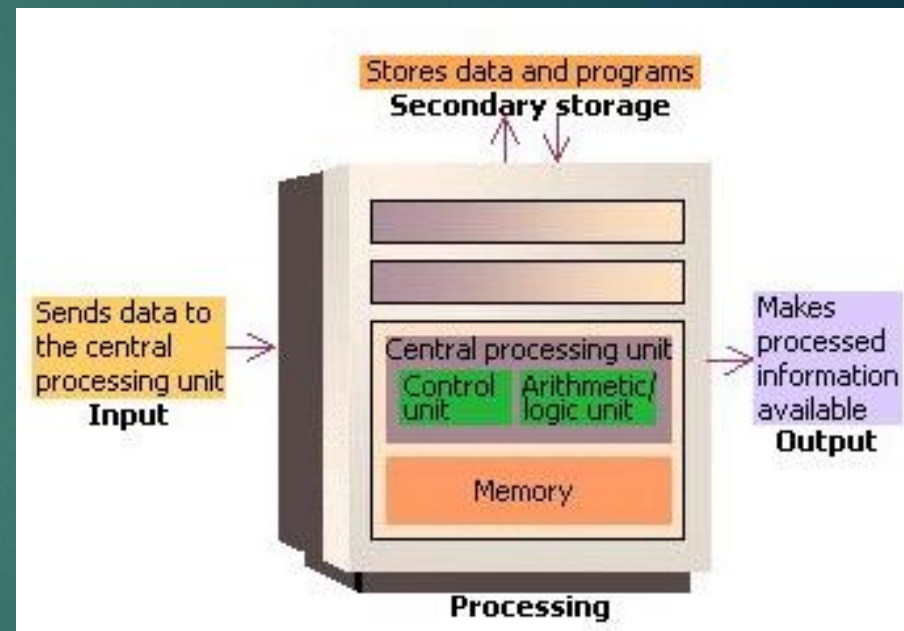


Central Processing Unit

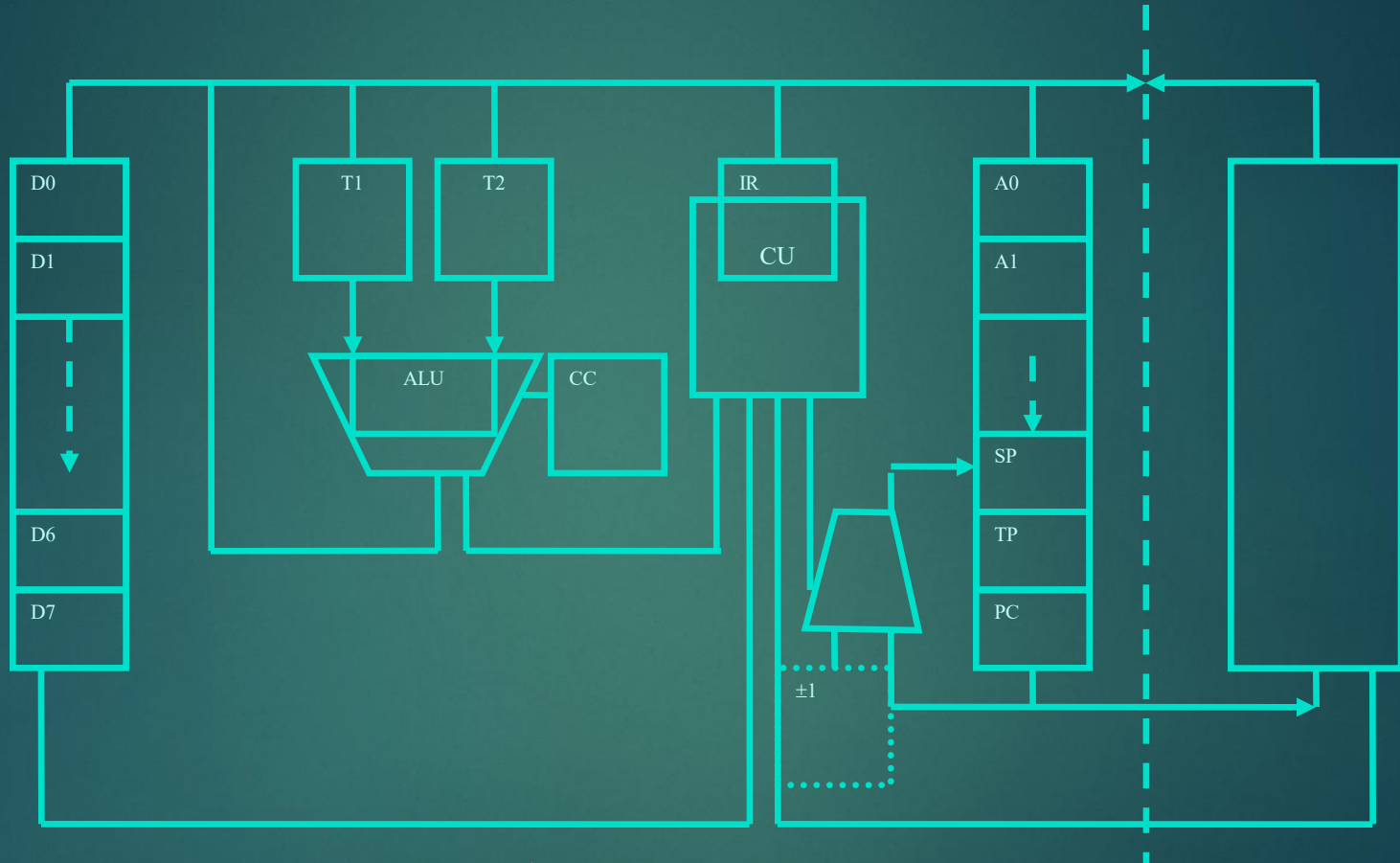
CPU (Central Processing Unit) is the “brain” of the computer. Its function is to execute programs stored in the main memory by fetching their instructions, examining them, and then executing them after another.

Parts of the CPU

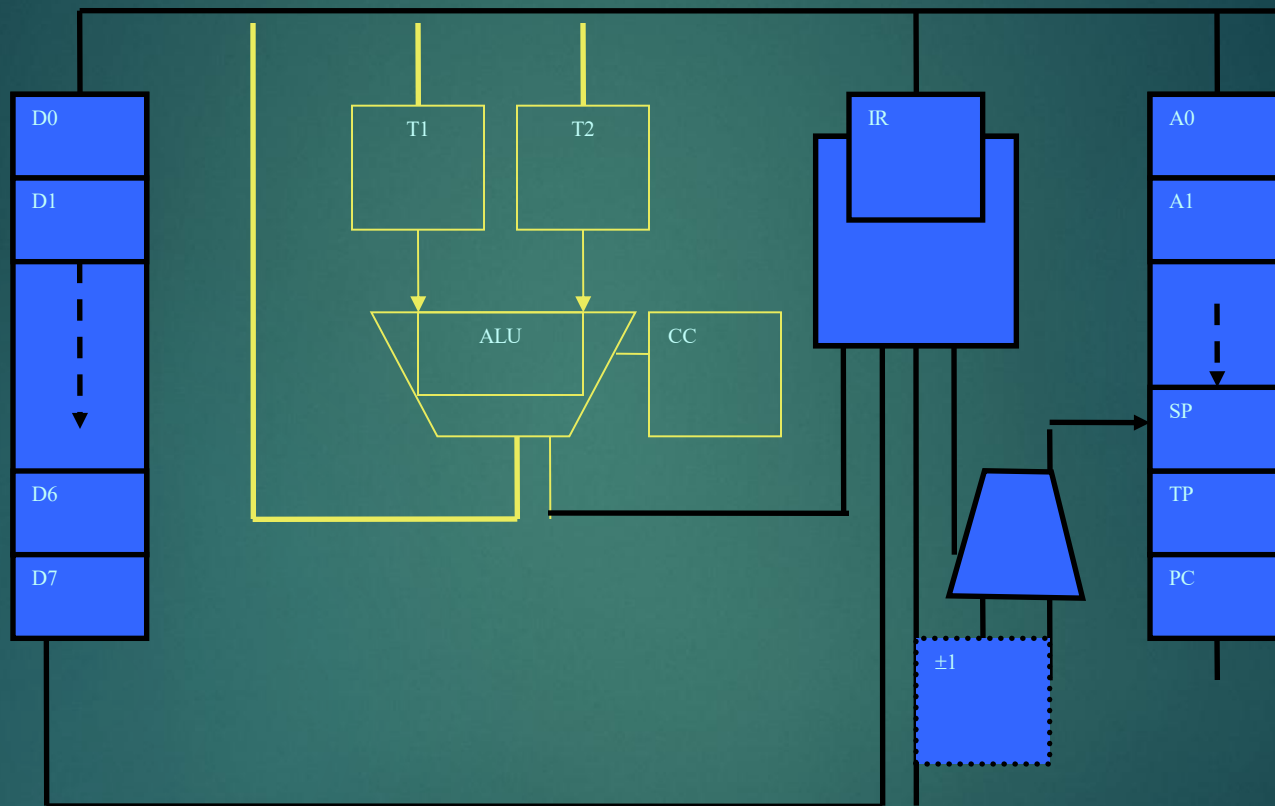
1. Arithmetic Logic Unit
2. High Speed Memory (Registers and cache)
3. Control unit and Instruction Register
4. Internal/ System Bus
5. Input/ Output Functions



Processor

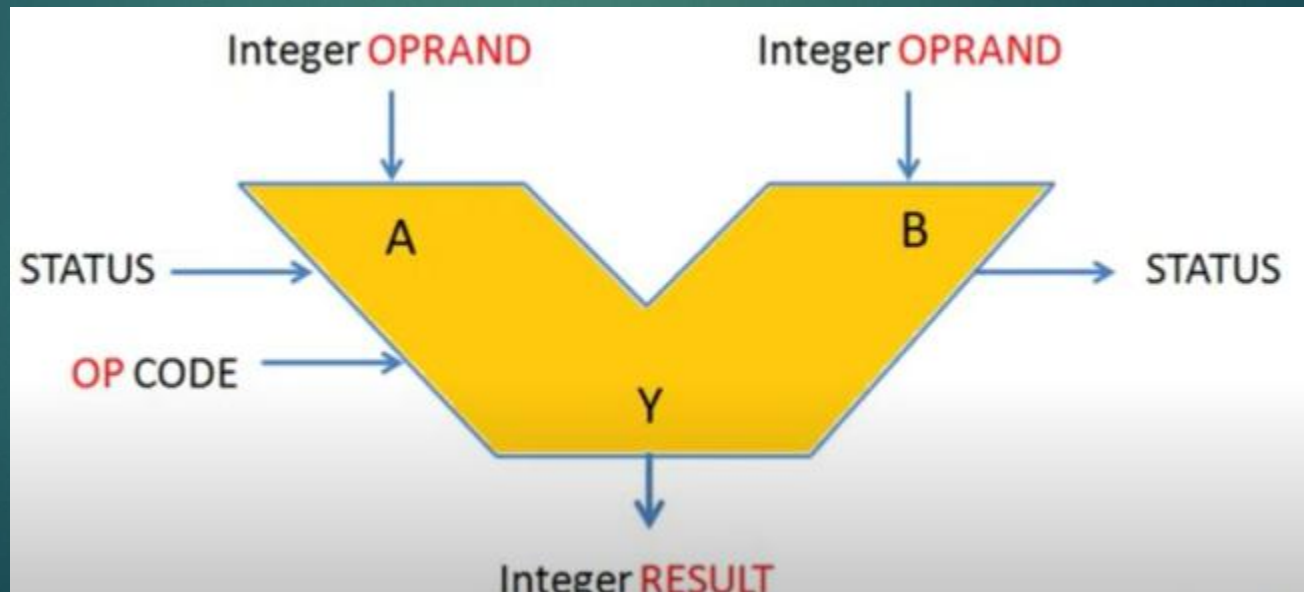


Central Processing Unit



Arithmetic Logic Unit

- ▶ ALU performs operations such as addition and BOOLEAN needed to carry out the instructions
- ▶ It is the fundamental building block of the processor
- ▶ responsible for carrying out the calculations and logical comparisons required for the execution of programs
- ▶ takes two input operands, performs the specified operation, and produces an output result, and stores it in the output register

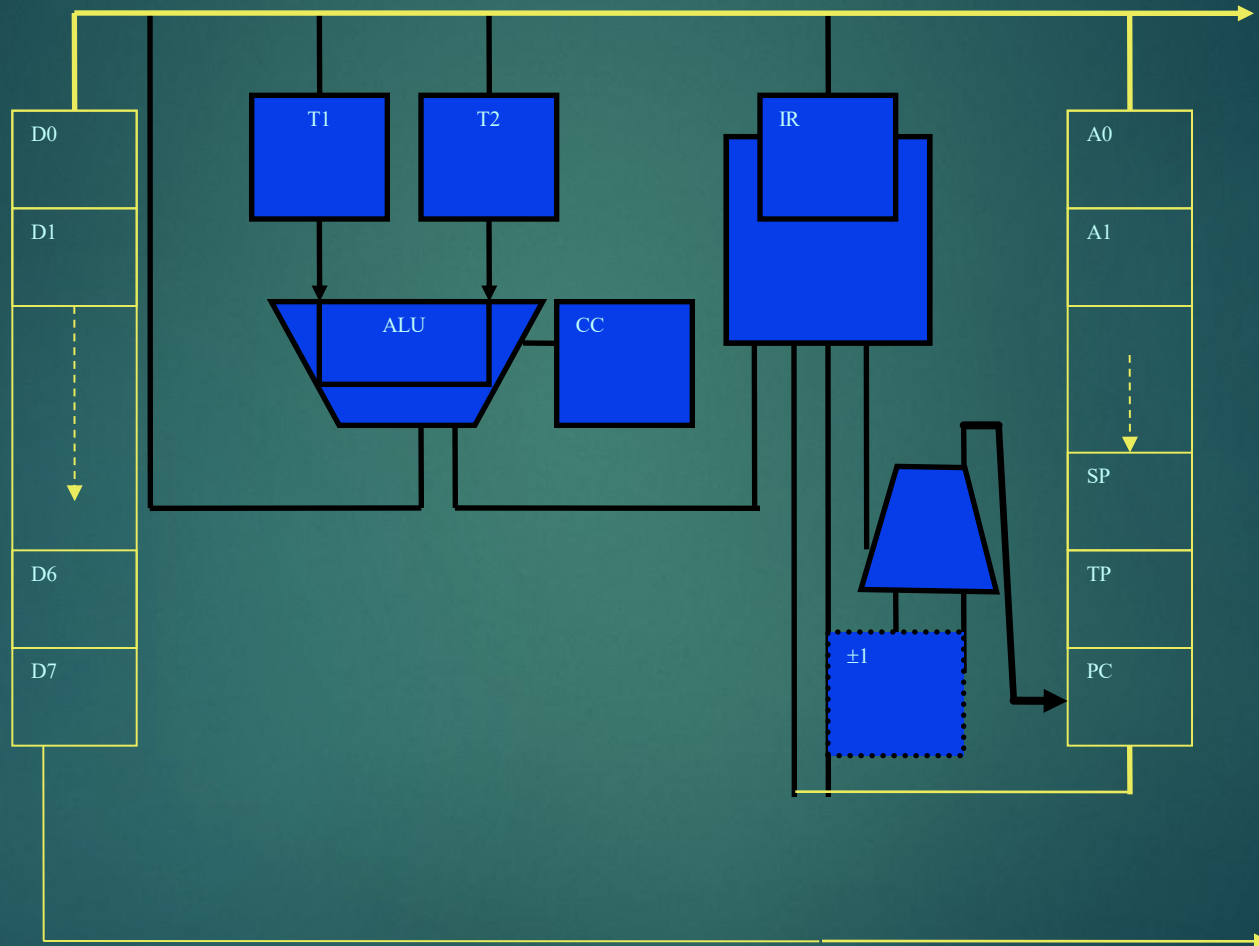


Parts of the CPU



2. **High Speed Memory** – used to store temporary results and certain control information. It consists of a number of registers, each of which has a certain function. The most important register is the program counter, which points to the next instruction to be executed.

High Speed Memory



Memory

~~RAM~~ ~~cont~~ ~~at~~ ~~of~~ ~~the~~
~~oper~~ ~~the~~ ~~prog~~ ~~ram~~
~~addressed~~

~~ES~~ ~~is~~ ~~in~~ ~~direct~~ ~~mode~~
~~by~~ ~~the~~ ~~simple~~
~~pointer~~

Why binary digits, NOT decimal?

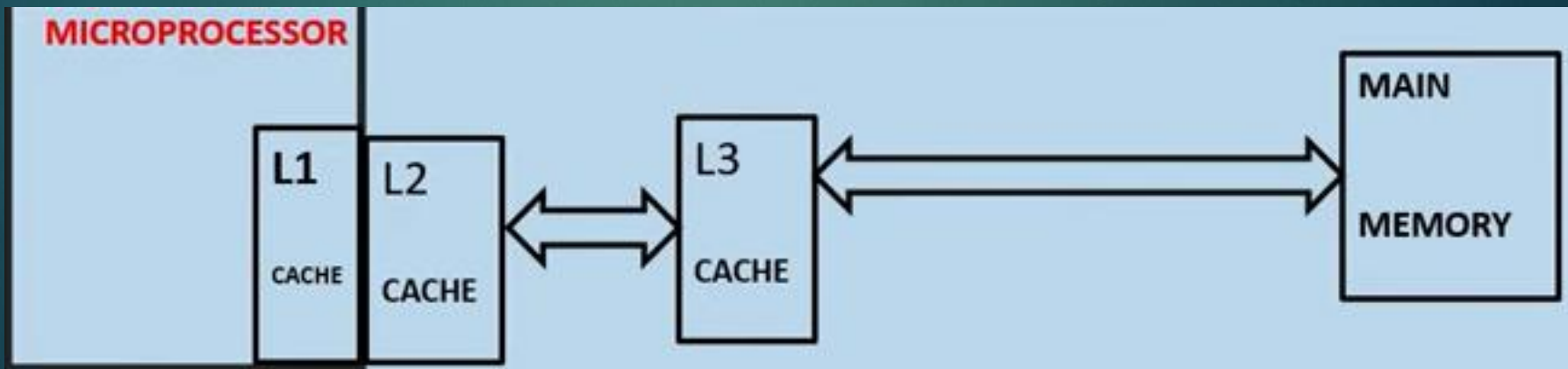
- 1 Digital information can be stored by distinguishing between different levels of some continuous physical quantity, such as voltage or current
 - The more values to be distinguished, the less separation between adjacent values, the less reliable the memory

Why binary digits, NOT decimal?

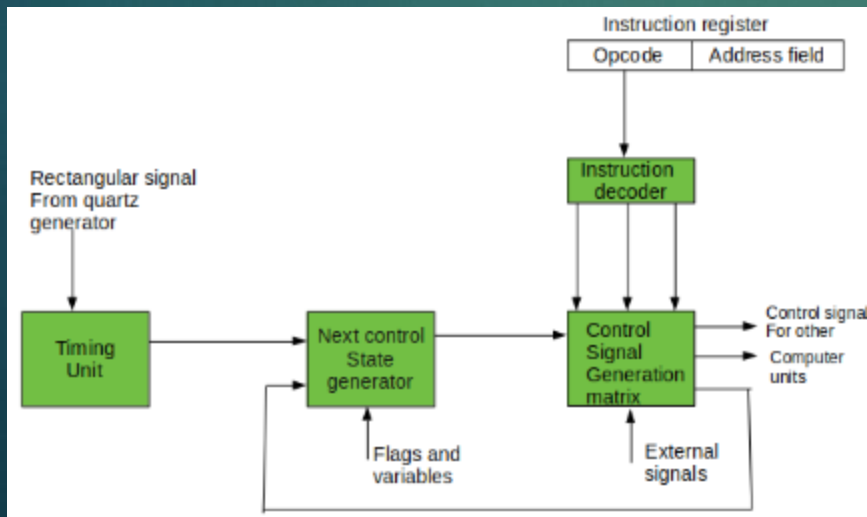
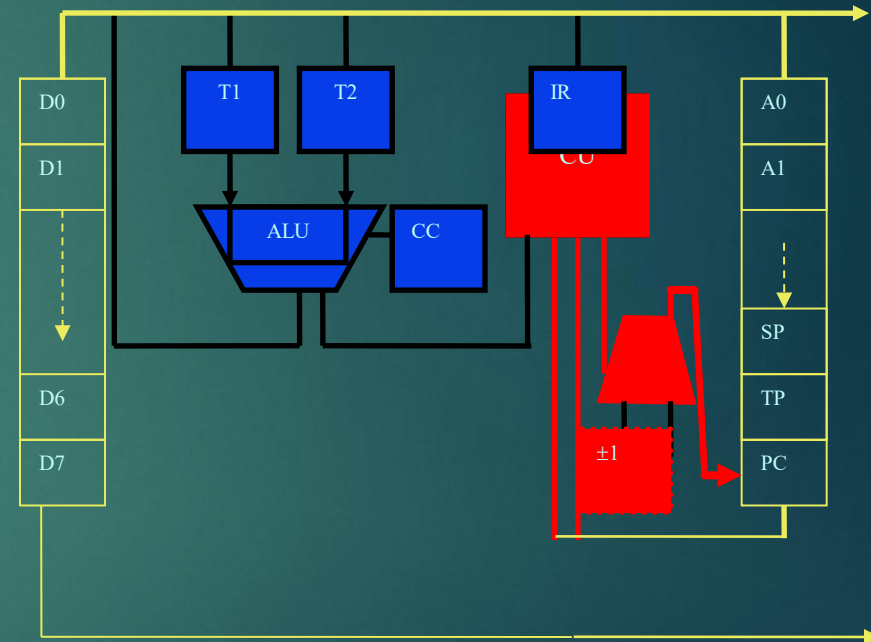
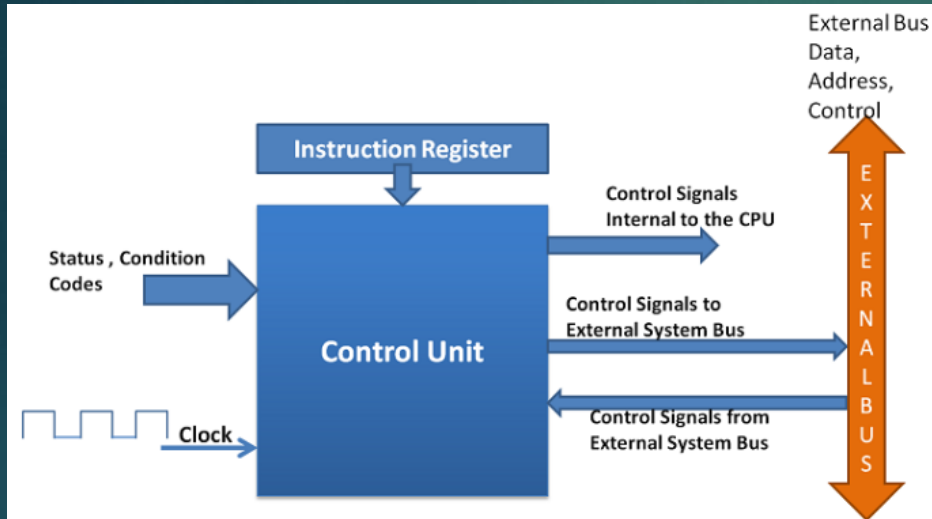
2 When using decimal digits you need, say four bits to store one decimal digit (from 0-9) – this implies that it will provide 16 combinations (2⁴) with six combinations not used

Cache Memory

- ▶ High speed small amount of memory inside the microprocessor
- ▶ CPU quickly access this memory as compared to RAM
- ▶ Stores data that is frequently required by CPU – that is, it keeps active portion of main memory

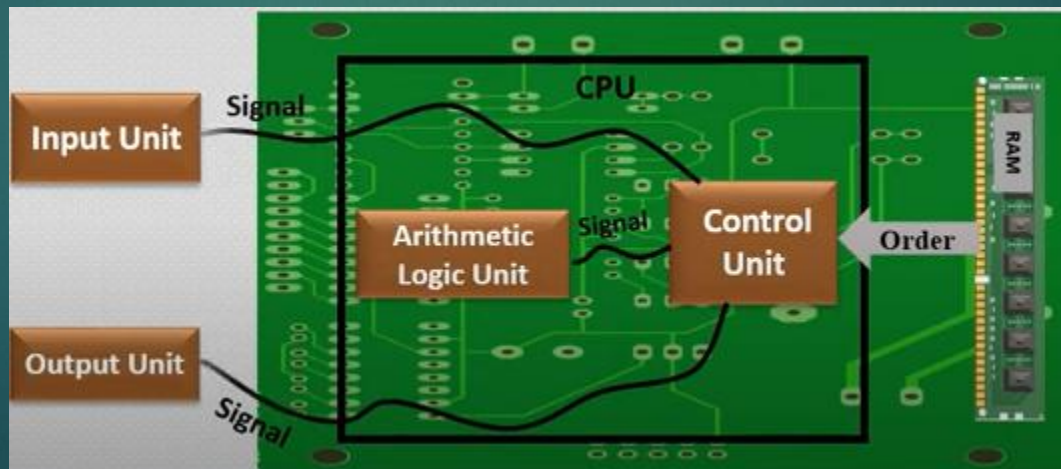


Control Unit Model



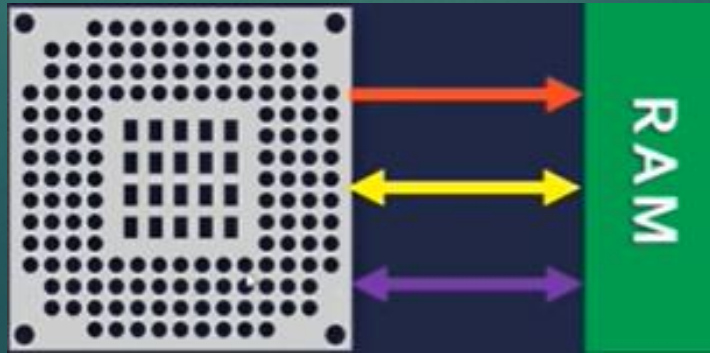
Function of Control Unit

- ▶ For each operation, a unique code is provided (e.g. ADD, MOVE)
- ▶ A hardware segment accepts the code and issues the control signals – tells the ALU what operation to perform on data
 - ▶ Task - receive instruction from the memory unit (ROM or RAM)
 - ▶ Convert it to appropriate signals, that make the input and output units, ALU operate as required



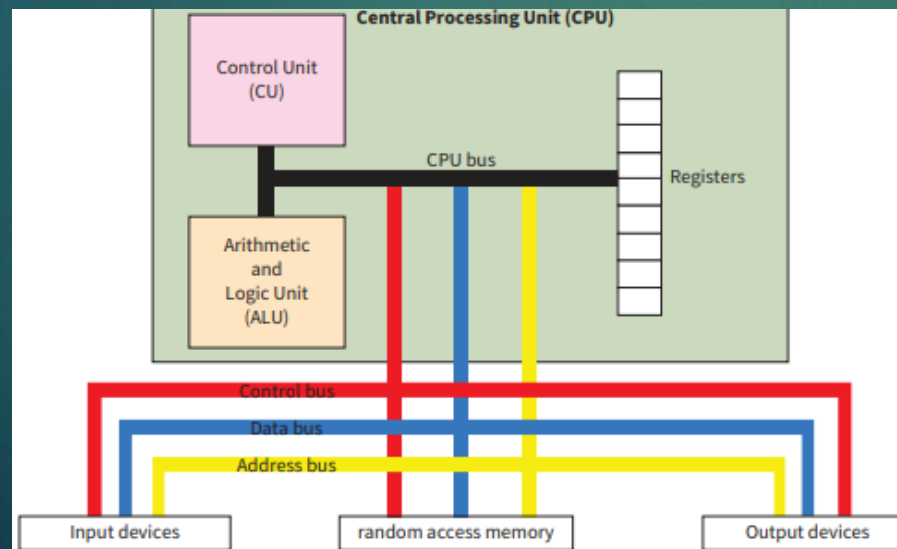
Internal CPU Bus

- ▶ A group of parallel wires used to carry data from one part of the computer to another part of the computer
- ▶ Connected all the devices in the CPU
- ▶ Speed – megahertz
- ▶ Size – no. of bits to transfer at a time (64 bit or 64-bit width)
- ▶ 3 types of buses inside the CPU
 - ▶ **Address bus** sends a memory address along the bus from the CPU to the memory
 - ▶ **Data bus** sends the actual data to and from the memory
 - ▶ **Control bus** carries commands from the CPU and status messages from other hardware devices



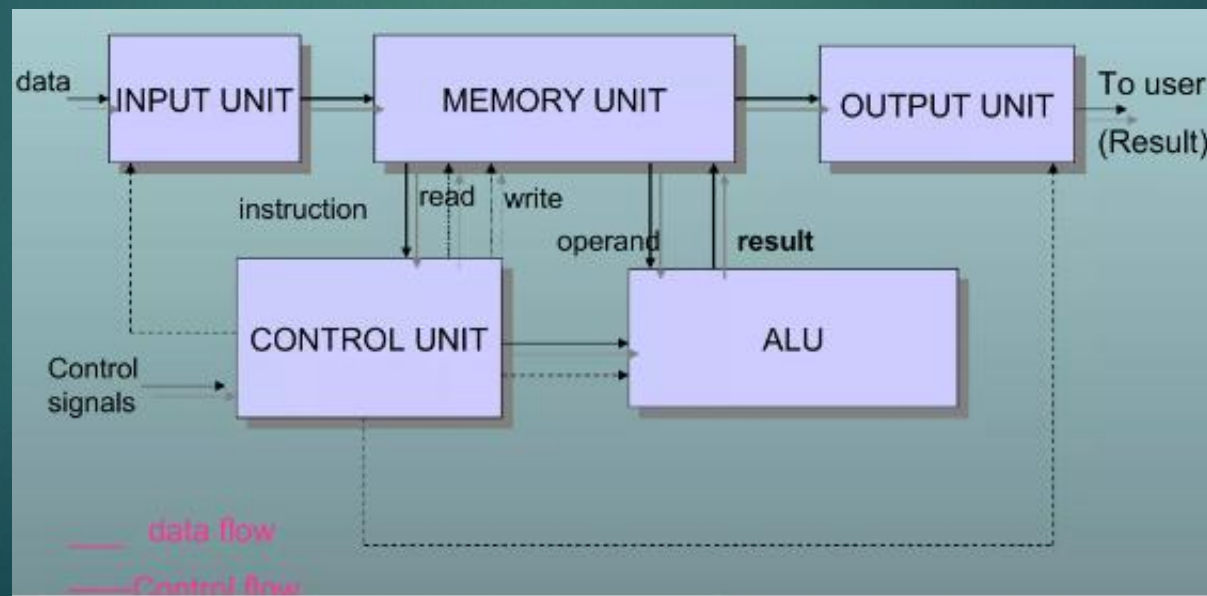
CPU Bus (continuation)

- ▶ The control bus connects the control unit (CU) with the other components of the CPU and devices in the computer system.
- ▶ The control unit uses it to send instructions to other components of the computer.
- ▶ The components use a bus to send information back to the CPU as well.
- ▶ The data bus is used for the transfer of data between the CPU and the RAM and the address bus for the CPU to access memory locations in the main memory.



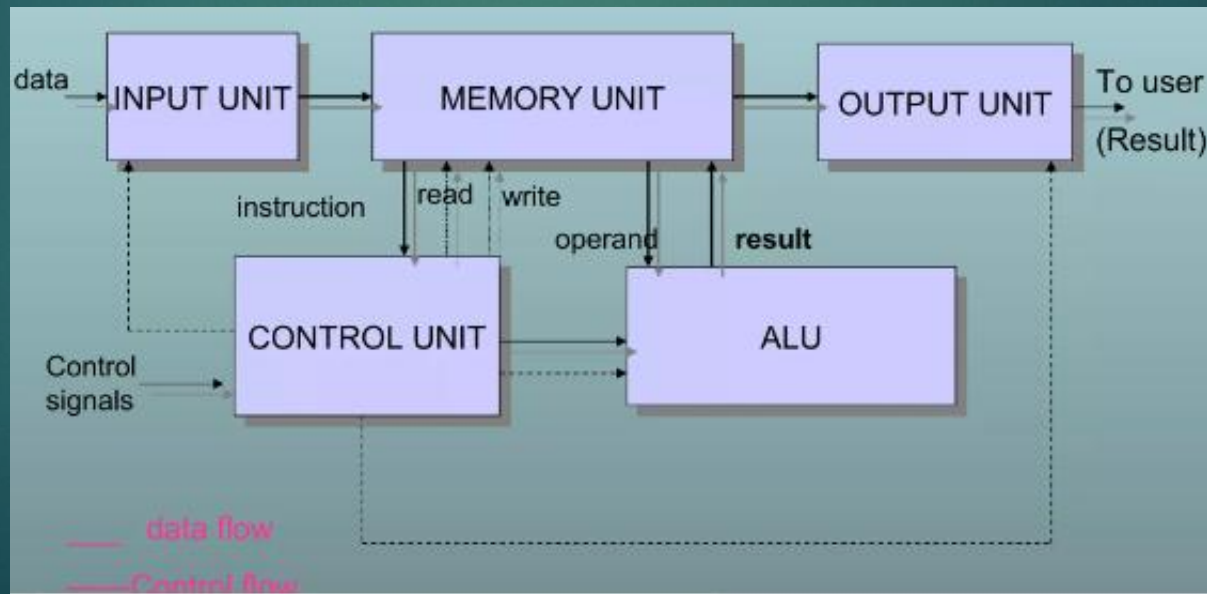
Input/ Output Functions

- ▶ Data is supplied to the computer through an input device.
- ▶ The data is then transferred to memory under the supervision of the control unit.
- ▶ Each instruction is interpreted before execution
- ▶ Control unit informs the ALU of the precise operation to be performed

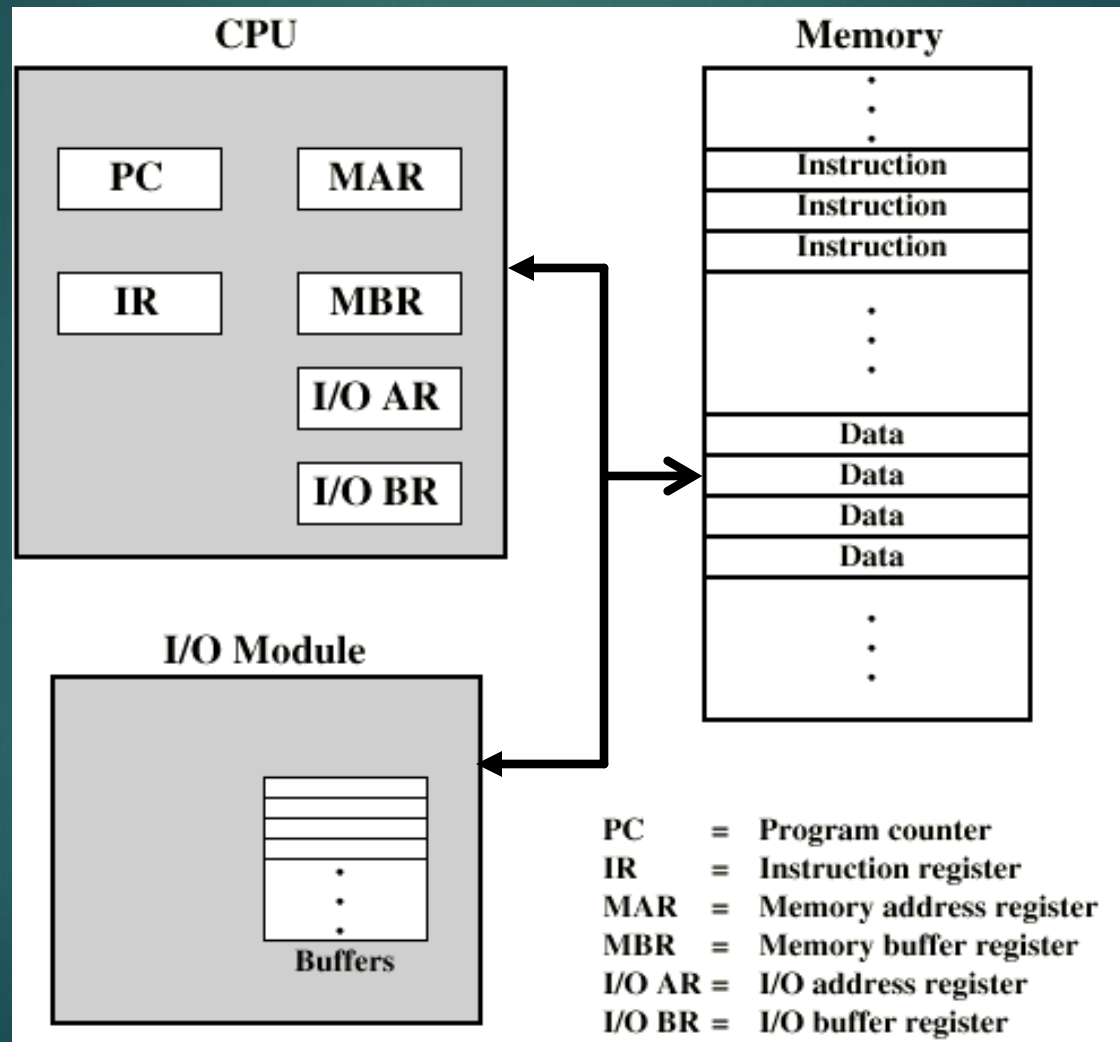


Input/ Output Functions (cont'n)

- ▶ Directs the transfer of operands to the ALU
- ▶ The ALU then performs all calculations and comparisons needed.
- ▶ And the results are then passed unto the memory unit. From there to the output unit

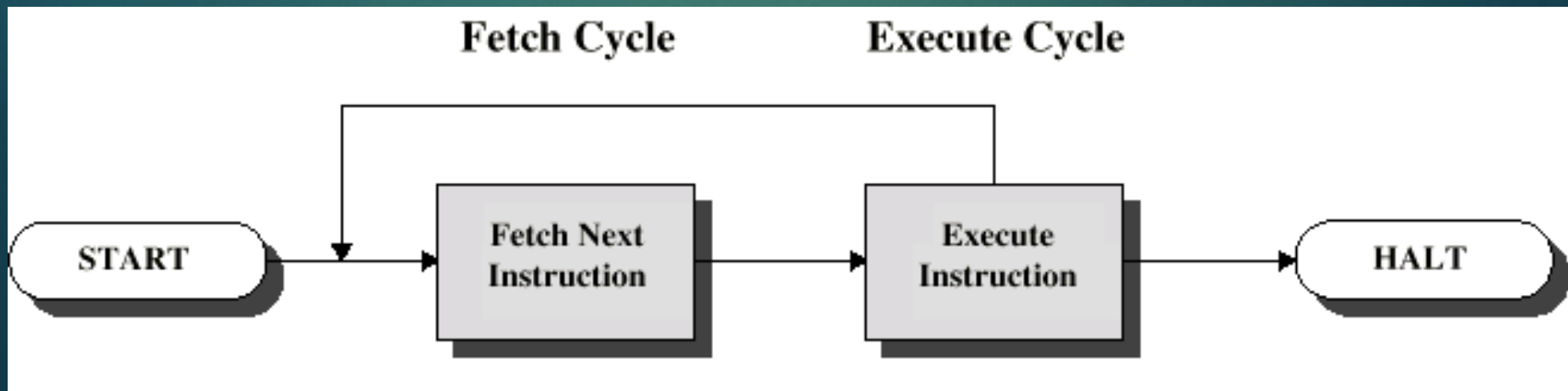


Computer Components: Top Level View



Instruction Cycle

- ▶ Two steps:
 - ▶ Fetch – the basic operation (instruction) cycle of a computer
 - ▶ Execute – the computer retrieves a program instruction from its memory. It then establishes and carries out the actions that are required for that instruction.
- ▶ The cycle of fetching, decoding and executing an instruction is continually repeated by the CPU when the computer is turned on.



Fetch Cycle

- ▶ Program Counter (PC) holds address of next instruction to fetch
- ▶ Processor fetches instruction from memory location pointed to by PC
- ▶ Increment PC (Unless told otherwise)
- ▶ Instruction loaded into Instruction Register (IR)
- ▶ Processor interprets instruction and performs required actions

Random access memory

The program instructions and the data to be used are stored in RAM.

Address bus

Data bus

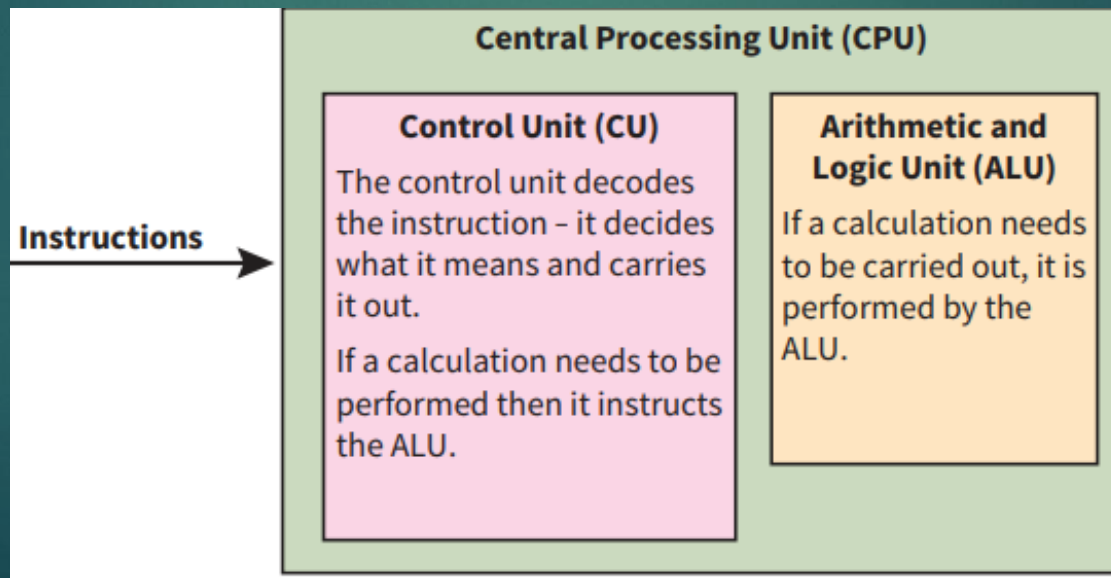
Central Processing Unit (CPU)

The CPU sends a signal to the RAM requesting the next instruction or the data to be used.

The instruction or data are sent to the CPU.

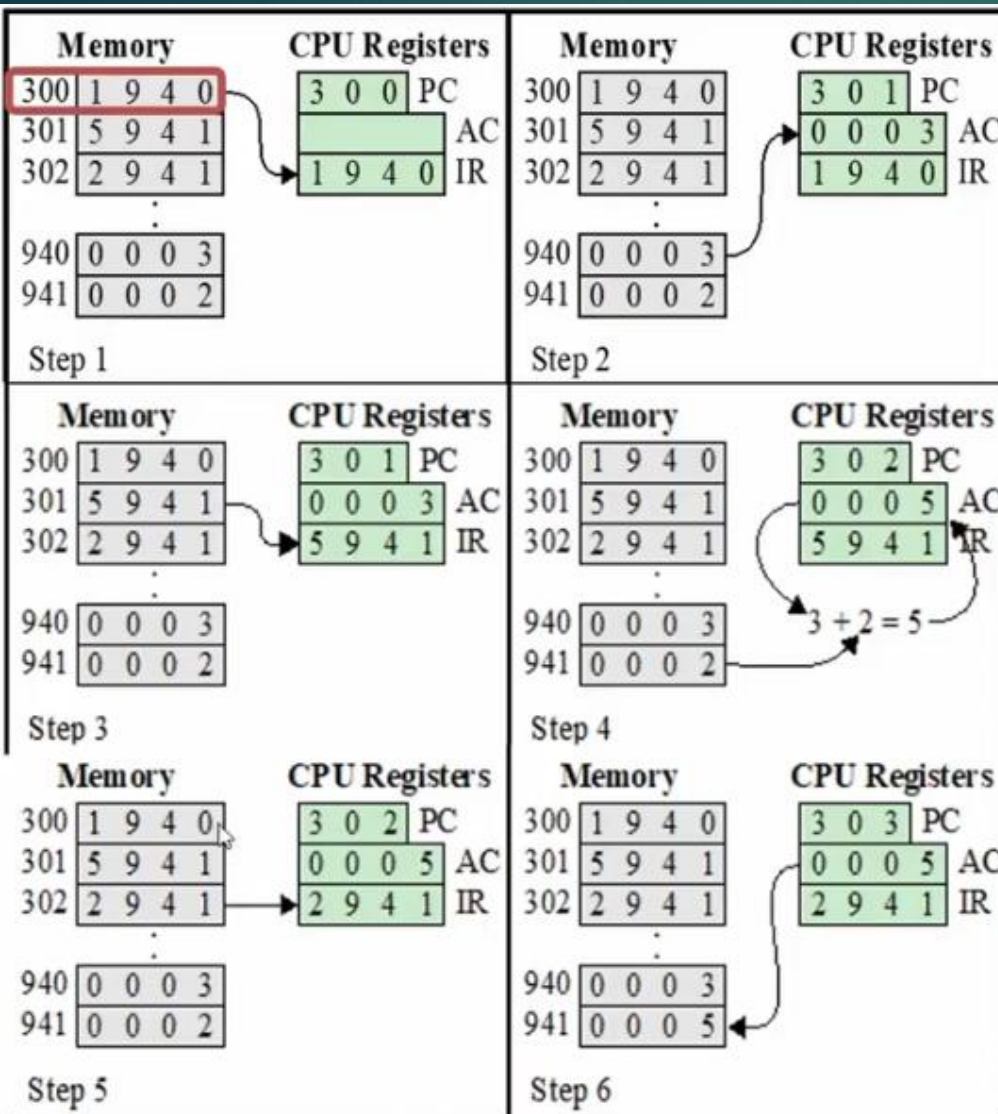
Execute Cycle

- ▶ Processor-memory: data transfer between CPU and main memory
- ▶ Processor I/O: Data transfer between CPU and I/O module
- ▶ Data processing: Some arithmetic or logical operation on data
- ▶ Control: Alteration of sequence of operations (e.g. jump)
- ▶ Combination of above



Step	Fetch execute cycle steps	Simplified description
1	The PC contains the address of the memory location that has the next instruction which has to be fetched	PC has address of next instruction
2	This address is then copied from the PC to the MAR via the address bus	PC copied to the MAR
3	The contents (instruction) at the memory location (address) contained in MAR are then copied into the MDR	Lookup MAR and get contents. Copy contents into the MDR
4	The contents (instruction) in the MDR is then copied and placed into the CIR	Copy MDR contents into the CIR
5	The value in the PC is then incremented by 1 so that it now points to the next instruction which has to be fetched	PC is then incremented by 1
6	The instruction is finally decoded and then executed by sending out signals (via control bus) to the various components of the computer	The instruction is decoded and then executed
7	Repeat	

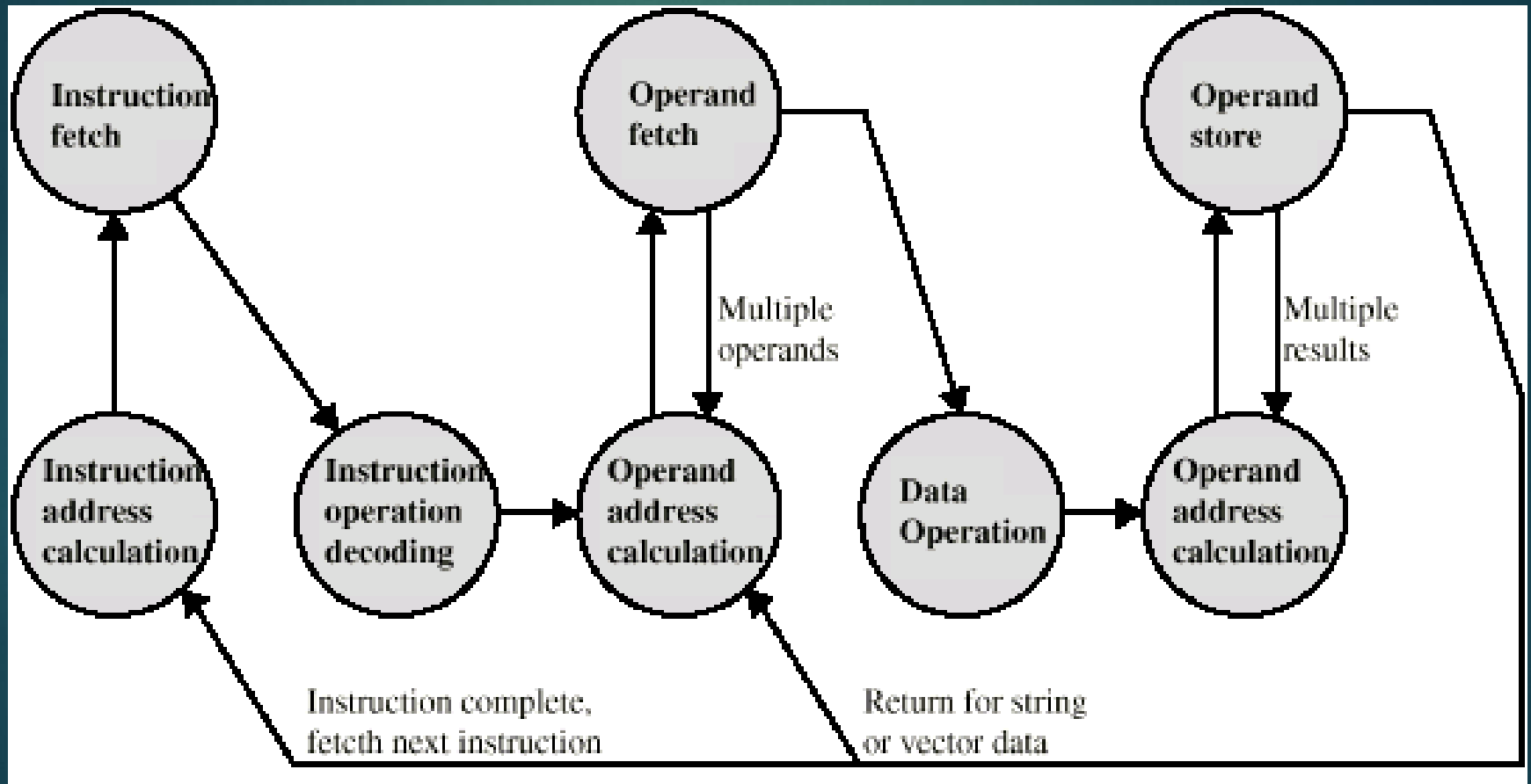
Example of Program Execution



- 1 0001 = Load AC from Memory
- 2 0010 = Store AC to Memory
- 5 0101 = Add to AC from Memory

Op-Code	Addr.	Meaning
1(0001)	940	Ac=(940)=0003
5(0101)	941	AC=AC+(941) Ac=3+2=5
2(0010)	941	Addr(941)=AC=5

Instruction Cycle - State Diagram



Interrupts

- ▶ A signal from a device attached to a computer or from a program within the computer that requires the operating system to figure out what to do next.
- ▶ Needs of Interrupts?
 - ▶ Devices and programs occasionally need CPU service, but we can't predict it
 - ▶ For interaction with CPU, each device or a program is allowed to give interrupt so that it can be used as a signal to the processor
 - ▶ Need a way for the CPU to find out devices/programs need attention

Two types of Interrupt

- ▶ Hardware Interrupt – If the signal for the processor is from external device or hardware
 - ▶ Example: Keystroke (pressing of key on keyboard and mouse movements)
- ▶ Software Interrupt – caused either by special instruction in the instruction set or by an exceptional condition in the processor itself
 - ▶ Example: it is generated when divide a number by zero, accessing unavailable memory

Hardware Interrupt

- ▶ Mask-able Interrupt: the hardware interrupts which can be delayed when a much higher priority interrupt has occurred to the processor
 - ▶ Example: keyboard interrupt, mouse, etc
- ▶ Non Mask-able Interrupt: The hardware which cannot be delayed and should be processed by the processor immediately
 - ▶ Example: hardware failure, power failure

Software Interrupt Types

- ▶ Normal Interrupts: The interrupts which are caused by the software instructions
 - ▶ Example: system call such as opening a file, printing a message, or allocating memory.
- ▶ Exception: Unplanned interrupts while executing a program
 - ▶ example: while executing a program if we got a value which should be divided by zero, overflow, or invalid opcode

Interrupts versus Procedures

Interrupts

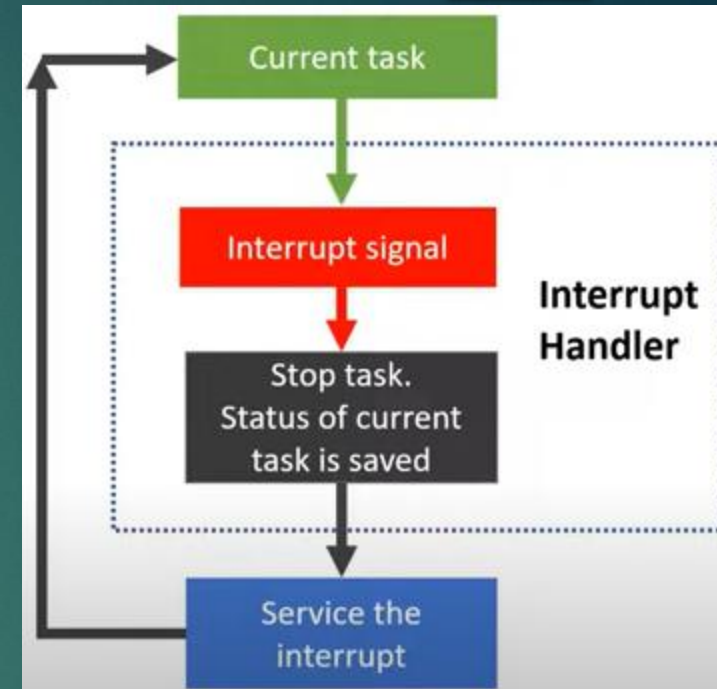
- Initiated by both *software* and *hardware*
- Can handle *anticipated* and *unanticipated* internal as well as external events
- ISRs or interrupt handlers are memory resident
- Use numbers to identify an interrupt service
- (E)FLAGS register is saved automatically

Procedures

- Can only be initiated by *software*
- Can handle *anticipated* events that are coded into the program
- Typically loaded along with the program
- Use meaningful names to indicate their function
- Do not save the (E)FLAGS register

Interrupt handling (micro- controller

- ▶ Interrupt: After receiving an interrupt signal, the microcontroller stops whatever it is doing and serves the device.
 - ▶ Program associated with the interrupt is called the interrupt service routine (ISR) or interrupt handler.
- ▶ Polling: Another interrupt handler – in this, the microcontroller continuously monitors the status of the given devices.
 - ▶ When the conditions are met, (that is, if the device needs some service) it performs the service. Next, it moves on to monitor the next device. It continues in this fashion until everyone is serviced.

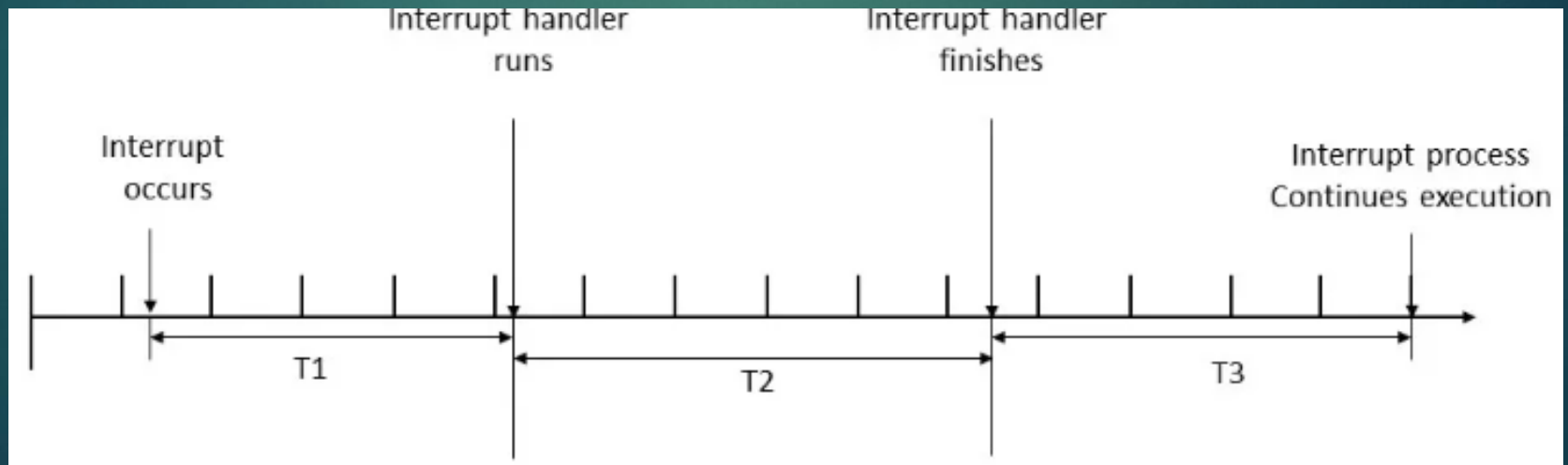


Interrupt VS Polling

- ▶ Interrupt is preferred over polling for the following reasons:
 - ▶ Polling is not efficient, because it wastes much of the microcontroller's time by checking devices that do not need service.
 - ▶ Polling don't have priority, only on round robin basis they can get the attention of the microcontroller.
- ▶ Interrupt can serve many devices; and each device can get the service of microcontroller based on priority.
 - ▶ In interrupt, a microcontroller can also ignore a device's request for service, this is not possible in the polling method

Interrupt Handling Process Timeline

- ▶ T1 is interrupt latency
- ▶ T2 is interrupt processing time
- ▶ T3 is interrupt termination time



References

- ▶ William Stallings, Computer Architecture and Organization. 11th Edition, 2019
- ▶ Structured Computer Organization by Andrew Tannenbaum
- ▶ <https://www.slideshare.net/UrwaShanza1/5-interput-46-47-1#7>